

T.C.
BANDIRMA ONYEDİ EYLÜL ÜNİVERSİTESİ
MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ (İNGİLİZCE) BÖLÜMÜ

2025 - 2026 BMI3245 DATA SECURITY

GÜVENLİ GİRİŞ VE HESAP KORUMA SİSTEMİ

Sümeyya ÖZTÜRK
2311504280

Bandırma, 2026

İÇİNDEKİLER

1. Problem Tanımı…….. 3
2. Sistemin Gerçek Hayattaki Karşılığı……..…... 3
3. Projenin Data Security Dersi ile İlişkisi……..…….. 3
4. Kullanılan Güvenlik Kavramları…… 4
5. Kullanılan Algoritmalar ve Yöntemler……..... 4
6. Gizlilik, Bütünlük ve Kimlik Doğrulama……..… 6
7. Sistem Mimarisi……. 7
8. Kullanılan Veri Türleri…….…. 10
9. Kullanılan Hash ve Erişim Yapısı……..…… 10
10. Saldırı ve Hata Senaryoları……..…….. 12
11. Güvenlik Testleri……... 14
12. Uygulama Çıktıları…….…... 14
13. Sonuçlar ve Değerlendirme…….….. 16
14. Kaynakça……... 17

1. Problem Tanımı

Günümüzde dijital sistemlerin hızla yaygınlaşmasıyla birlikte kullanıcı hesaplarının güvenliği büyük önem kazanmıştır. Birçok uygulama kullanıcı adı, şifre ve kişisel bilgiler gibi hassas verileri saklamaktadır. Eğer bu sistemlerde yeterli güvenlik önlemleri alınmazsa saldırganlar kullanıcı hesaplarına yetkisiz erişim sağlayabilir ve önemli bilgileri ele geçirebilir.

Bu projenin amacı, Data Security dersinde öğrenilen güvenlik kavramlarını kullanarak güvenli bir giriş ve hesap koruma sistemi geliştirmektir. Sistem; güvenli şifre saklama, kullanıcı doğrulama, brute-force saldırılarına karşı koruma, güçlü şifre kontrolü ve loglama mekanizmalarını içermektedir.

Bu proje sayesinde gizlilik, bütünlük ve kimlik doğrulama prensiplerinin gerçek bir uygulamada nasıl kullanılabileceği gösterilmiştir.

Geleneksel giriş sistemlerinin bir kısmı kullanıcı şifrelerini düz metin olarak saklamakta veya yetersiz güvenlik yöntemleri kullanmaktadır. Bu durum aşağıdaki güvenlik problemlerine yol açmaktadır:

- Şifrelerin çalınması
- Brute-force saldırıları
- Yetkisiz erişim
- Zayıf şifre kullanımı
- Kullanıcı kimliğinin taklit edilmesi

Bu projede ele alınan temel problem, kullanıcı hesaplarının yaygın saldırılara karşı korunması ve güvenli kullanıcı doğrulama mekanizmasının oluşturulmasıdır.

2. Sistemin Gerçek Hayattaki Karşılığı

Bu projede geliştirilen güvenli giriş sistemi gerçek hayatta birçok alanda kullanılmaktadır:

- Bankacılık sistemleri
- Sosyal medya platformları
- Üniversite otomasyon sistemleri
- E-ticaret siteleri
- Hastane bilgi sistemleri
- Devlet uygulamaları

Bu proje gerçek sistemlerde kullanılan temel güvenlik mekanizmalarının küçük ölçekli bir simülasyonudur.

3. Projenin Data Security Dersi ile İlişkisi

Bu proje Data Security dersi kapsamında öğrenilen birçok güvenlik kavramını uygulamalı olarak içermektedir:

- Kriptografik hash algoritmaları
- Kimlik doğrulama
- Erişim kontrolü
- Güvenlik loglama
- Saldırı önleme
- Şifre güvenliği
- Gizlilik ve bütünlük

Bu sayede teorik güvenlik bilgilerinin gerçek bir yazılım sistemine nasıl uygulanabileceği gösterilmiştir.

4. Kullanılan Güvenlik Kavramları

Projede aşağıdaki güvenlik kavramları kullanılmıştır:

Güvenlik Kavramı	Kullanım Amacı
SHA-256 Hash	Şifrelerin güvenli saklanması
Kimlik Doğrulama	Kullanıcı doğrulama işlemi
Erişim Kontrolü	Yetkisiz erişimi önleme
Audit Log	Giriş hareketlerini kayıt altına alma
Brute Force Koruması	Çoklu yanlış girişleri engelleme
Güçlü Şifre Politikası	Güvenli parola oluşturma

5. Kullanılan Algoritmalar ve Yöntemler

5.1 SHA-256 Hash Algoritması

SHA-256 bir kriptografik hash algoritmasıdır. Girilen veriyi sabit uzunlukta bir hash değerine dönüştürür.

Hash işlemi aşağıdaki şekilde ifade edilir:

$H(x) = SHA\text{-}256(x)$

Bu projede kullanıcı şifreleri doğrudan saklanmamaktadır. Bunun yerine şifrelerin SHA-256 hash değeri tutulmaktadır.

Örnek:

Orijinal Şifre:
123456

Hash Değeri:
8d969eef6ecad3c29a3a629280e686cf

SHA-256 algoritmasının avantajları:

- Tek yönlü çalışır
- Geri çözülmesi zordur
- Şifre gizliliği sağlar

```
  {
    "username": "sumeyya",
    "password":
"8d68c09bb34f6cc9d2b8f017036f5986e5b8f30a8ad5841b67c4cf69d519e98a",
    "attempts": 3,
    "locked": true
  },
```

(Şekil 5.1 Hash Değeri Örneği)

5.2 Güçlü Şifre Kontrolü

Sistem kullanıcı şifrelerinin aşağıdaki özelliklere sahip olmasını zorunlu tutmaktadır:

- En az 8 karakter
- Büyük harf
- Küçük harf
- Sayı
- Özel karakter

Bu şartları sağlamayan şifreler kabul edilmemektedir.

```
// GÜÇLÜ ŞİFRE KONTROLÜ
function isStrongPassword(password) {

    const regex =
        /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&]).{8,}$/;

    return regex.test(password);
}
    if (!isStrongPassword(password)) {

        return res.json({
            success: false,
            message:
                "Şifre en az 8 karakter, büyük harf, küçük harf, sayı
ve özel karakter içermelidir!"
        });
    }
```

(Şekil 5.2 Güçlü Şifre Kontrolü)

5.3 Hesap Kilitleme Mekanizması

Brute-force saldırılarını önlemek amacıyla sistem 3 başarısız giriş denemesinden sonra hesabı kilitlemektedir.

Bu yöntem parola tahmin saldırılarını önemli ölçüde azaltmaktadır.

```
// 3 HATALI GİRİŞ
        if (user.attempts >= 3) {
            user.locked = true;
            writeLog(`${username} account locked`);
        }
       fs.writeFileSync(
            USERS_FILE,
            JSON.stringify(users, null, 2)
        );
       writeLog(`${username} failed login`);
            return res.json({
            success: false,
            message:
                `Hatalı Şifre! (${user.attempts}/3)`
        });
    }

 // HESAP KİLİTLİ
    if (user.locked) {

        writeLog(`${username} tried locked account`);

        return res.json({
            success: false,
            message: "Hesap kilitlendi!"
        });
    }
```

(Şekil 5.3 Hesap Kilitleme Mekanizması)

6. Gizlilik, Bütünlük ve Kimlik Doğrulama

6.1 Gizlilik

Gizlilik SHA-256 hash algoritması ile sağlanmaktadır. Kullanıcı şifreleri okunabilir şekilde saklanmamaktadır.

6.2 Bütünlük

Sistem kullanıcı bilgilerinin güvenli şekilde doğrulanmasını sağlayarak veri bütünlüğünü korumaktadır.

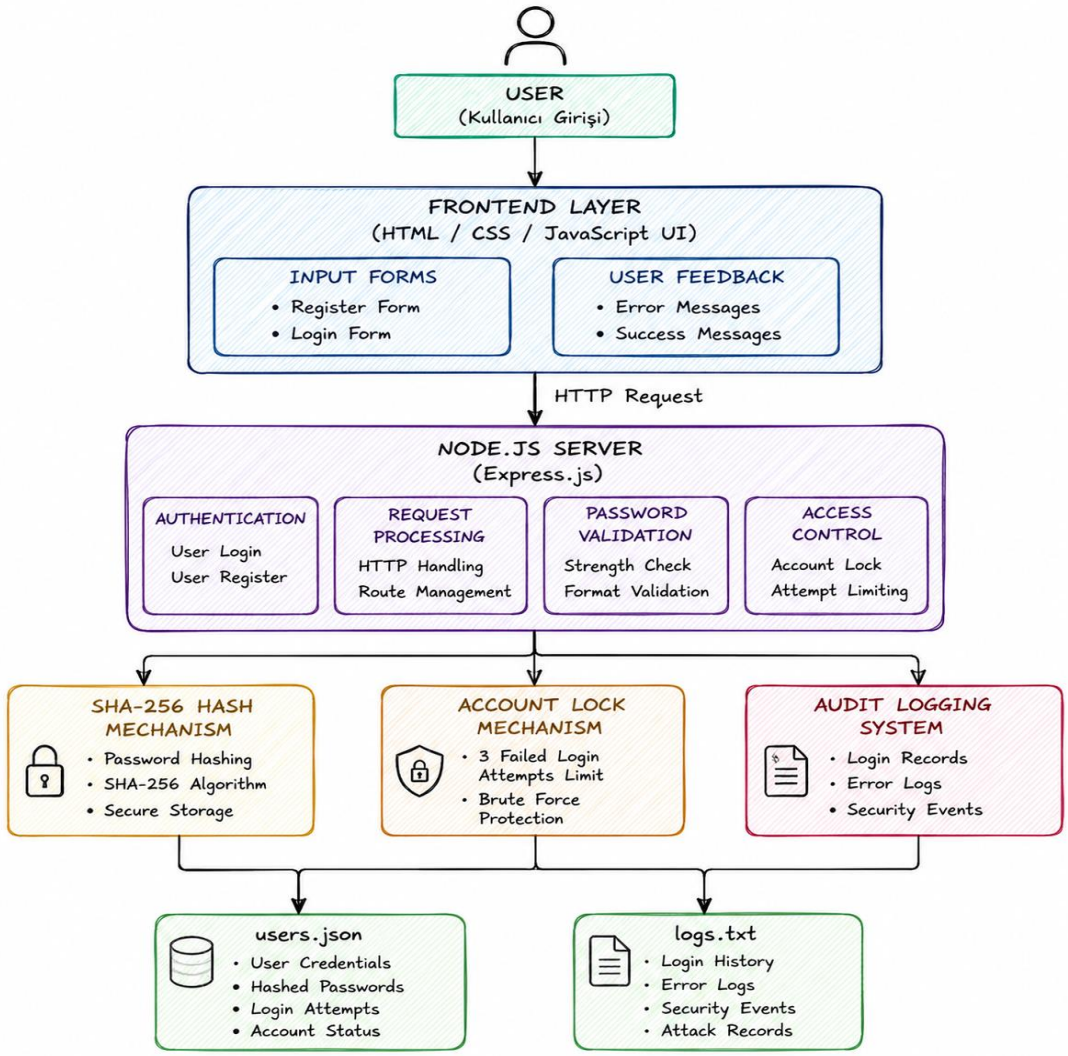
6.3 Kimlik Doğrulama

Kullanıcı adı ve şifre doğrulaması sayesinde yalnızca yetkili kullanıcıların sisteme erişmesine izin verilmektedir.

7. Sistem Mimarisi

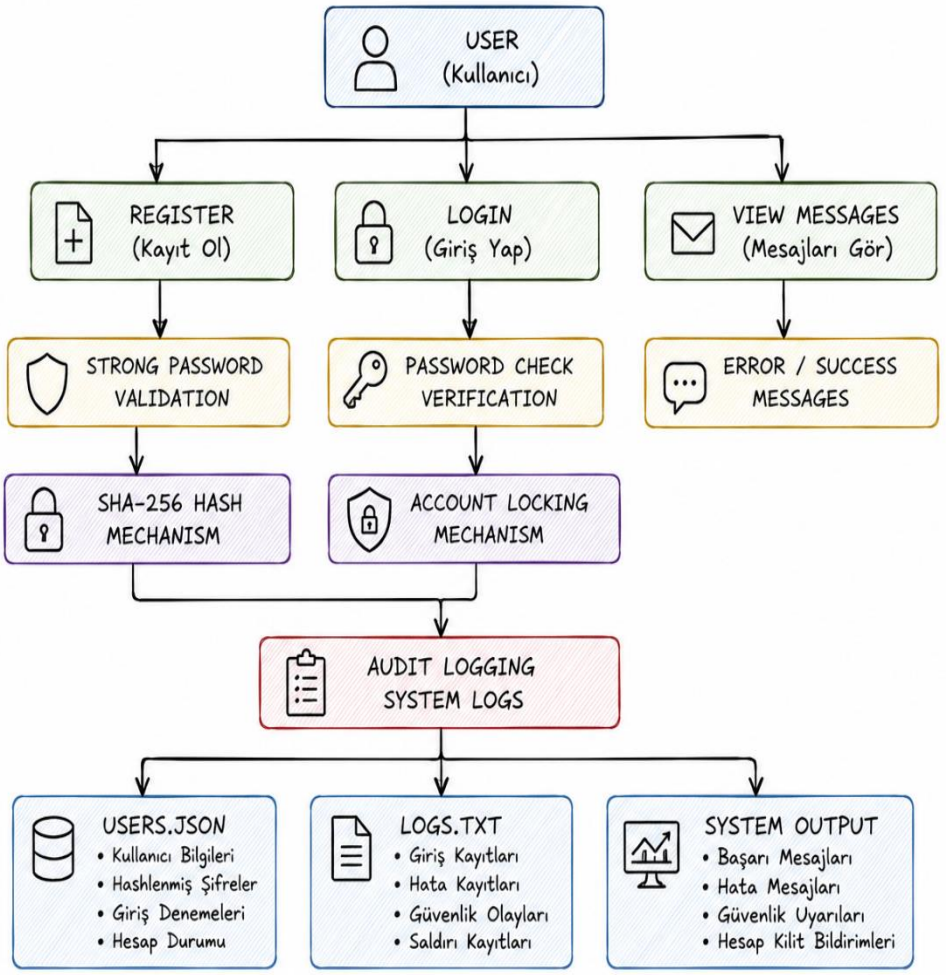
Sistem aşağıdaki bileşenlerden oluşmaktadır:

1. Kullanıcı Arayüzü
2. Sunucu Sistemi
3. Kullanıcı Veri Dosyası
4. Loglama Sistemi
5. Hash Mekanizması
6. Sistem çalışma akışı:
7. Kullanıcı giriş bilgilerini girer
8. Şifre hash işleminden geçirilir
9. Hash değeri kayıtlı veri ile karşılaştırılır
10. Doğrulama sonucu oluşturulur
11. Giriş işlemi log dosyasına kaydedilir



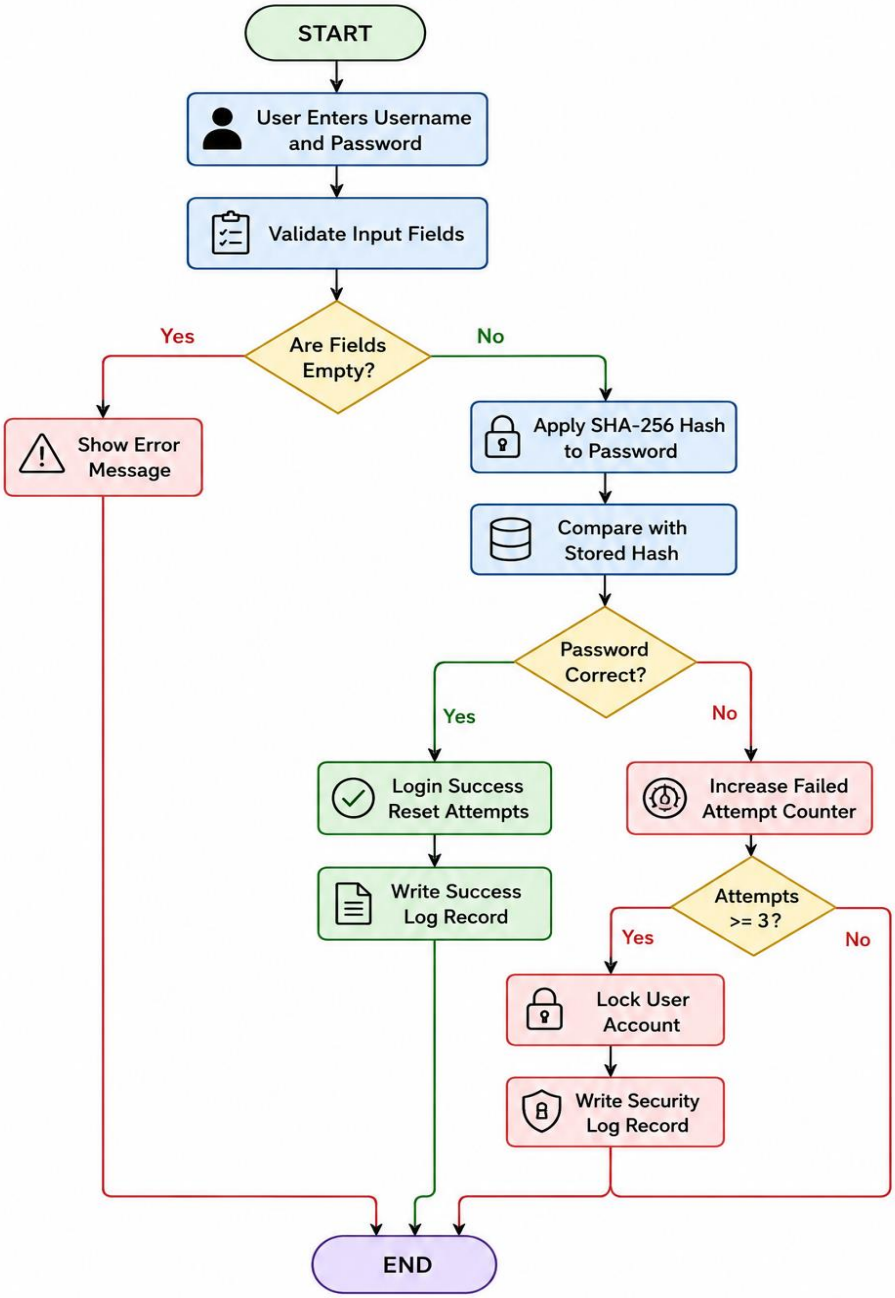
(Şekil 7.1 Sistem Mimarisi Diyagramı)

Şekil 7.1’de sistem mimarisi gösterilmektedir. Kullanıcı frontend arayüzü üzerinden sisteme kayıt olmakta veya giriş yapmaktadır. Girilen bilgiler Node.js sunucusuna gönderilmektedir. Sunucu tarafında SHA-256 algoritması ile şifre hash işlemi uygulanmakta ve kullanıcı doğrulaması gerçekleştirilmektedir. Başarısız giriş denemeleri kontrol edilerek brute-force saldırılarına karşı hesap kilitleme mekanizması çalıştırılmaktadır. Tüm giriş işlemleri ve güvenlik olayları log dosyasına kaydedilmektedir.



(Şekil 7.2 Use Case Diyagramı)

Şekil 7.2’de kullanım senaryosu diyagramı gösterilmektedir. Sistemin temel aktörü kullanıcıdır. Kullanıcı sisteme kayıt olabilir, giriş yapabilir ve sistem mesajlarını görüntüleyebilir. Kayıt işlemi sırasında güçlü şifre doğrulaması ve SHA-256 hash mekanizması uygulanmaktadır. Giriş işlemi sırasında kullanıcı doğrulaması yapılmakta, başarısız giriş denemelerinde hesap kilitleme mekanizması devreye girmektedir. Tüm işlemler güvenlik amacıyla audit log sistemine kaydedilmektedir.



(Şekil 7.3 Activity Diyagramı)

Şekil 7.3'de sistemin activity diagramı gösterilmektedir. Süreç kullanıcı tarafından kullanıcı adı ve şifre bilgilerinin girilmesi ile başlamaktadır. Sistem ilk olarak giriş alanlarının boş olup olmadığını kontrol etmektedir. Daha sonra parola SHA-256 algoritması ile hash işlemine tabi tutulmakta ve kayıtlı hash değeri ile karşılaştırılmaktadır. Eğer bilgiler doğruysa giriş başarılı olmakta ve log kaydı oluşturulmaktadır. Yanlış giriş durumunda başarısız deneme sayısı artırılmaktadır. Üç başarısız giriş sonrasında kullanıcı hesabı kilitlenmekte ve güvenlik logu oluşturulmaktadır.

8. Kullanılan Veri Türleri

Projede aşağıdaki veri türleri kullanılmıştır:

Veri Türü	Kullanım Amacı
Kullanıcı Adı	Kullanıcı tanımlama
Şifre	Kimlik doğrulama
Hash Değeri	Güvenli şifre saklama
Giriş Deneme Sayısı	Saldırı önleme
Log Kayıtları	Güvenlik takibi

9. Kullanılan Hash ve Erişim Yapısı

9.1 Hash Yapısı

Kullanıcı şifrelerinin SHA-256 hash değerleri kullanıcı veri dosyasında tutulmaktadır.

```
// SHA-256 hash function
function hashPassword(password) {
    return crypto
        .createHash("sha256")
        .update(password)
        .digest("hex");
}

 {
    "username": "sumeyya",
    "password":
"8d68c09bb34f6cc9d2b8f017036f5986e5b8f30a8ad5841b67c4cf69d519e98a",
    "attempts": 2,
    "locked": false
  },
  {
    "username": "rabia",
    "password":
"e6fcecae9c39ea1fdc98299811335908042390155c7e68fbf68c00b0fc0acfda",
    "attempts": 0,
    "locked": false
  }
```

(Şekil 9.1 Hash Yapısı ve Kullanıcı bilgilerinin tutulduğu users.json dosyası)

9.2 Erişim Yapısı

Yalnızca doğrulanmış kullanıcıların sisteme giriş yapmasına izin verilmektedir.

```
 const hashedPassword = hashPassword(password);

    if (hashedPassword === user.password) {

        user.attempts = 0;

        fs.writeFileSync(USERS_FILE, JSON.stringify(users, null, 2));

        writeLog(`${username} successful login`);

        return res.json({
            success: true,
            message: "Giriş Başarılı !"
        });
```

(Şekil 9.2 Kullanıcı giriş bilgilerinin test edilme mekanizması)

9.3 Loglama Yapısı

Tüm giriş denemeleri aşağıdaki bilgiler ile kaydedilmektedir:

- Tarih ve saat
- Kullanıcı adı
- Giriş sonucu

```
// Log yazma
function writeLog(message) {
    const time = new Date().toLocaleString();
    fs.appendFileSync(LOG_FILE, `[${time}] ${message}\n`);
}


[28.05.2026 16:45:02] sumeyya registered
[28.05.2026 16:45:30] sumeyya successful login
[28.05.2026 16:45:41] sumeyya failed login
[28.05.2026 16:45:52] sumeyya failed login
[28.05.2026 16:45:58] sumeyya account locked
[28.05.2026 16:45:58] sumeyya failed login
[28.05.2026 16:46:00] sumeyya tried to login to locked account
```

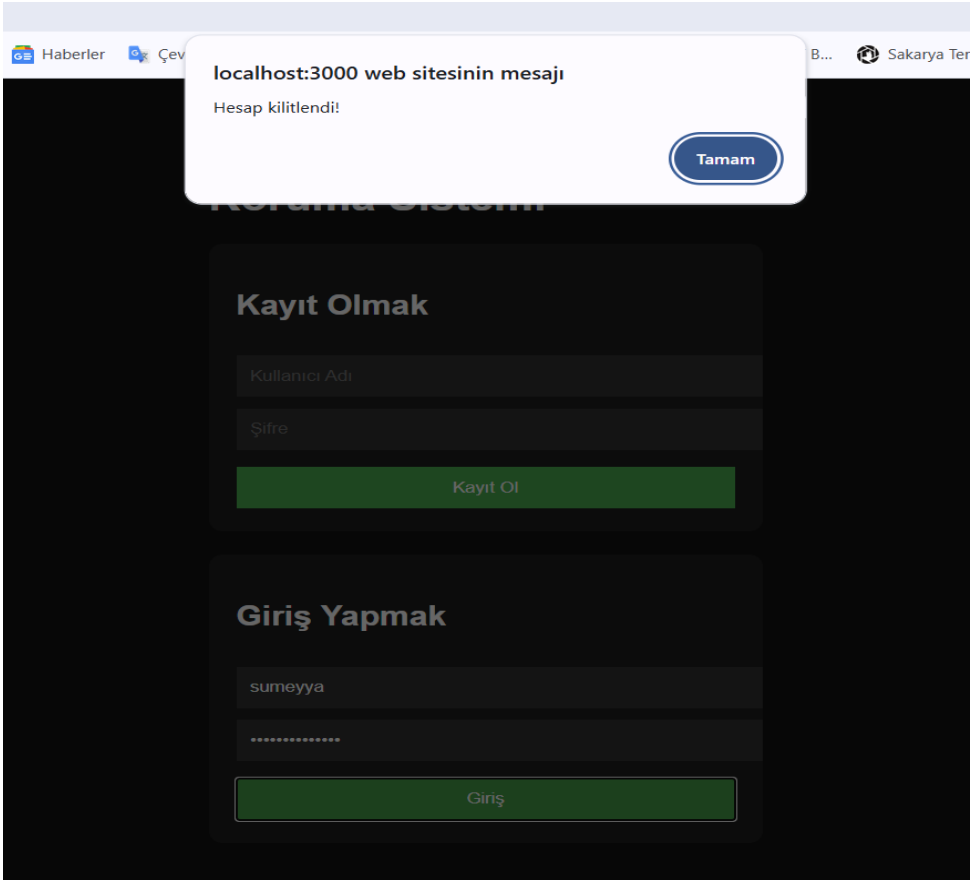
(Şekil 9.3 Kullanıcı giriş hareketlerinin kayıt edilmesi mekanizması)

10. Saldırı ve Hata Senaryoları

10.1 Brute Force Saldırısı

Senaryo:
Saldırgan art arda farklı şifreler deneyerek hesaba erişmeye çalışır.

Sistem Tepkisi:
Üç başarısız girişten sonra hesap kilitlenmektedir.

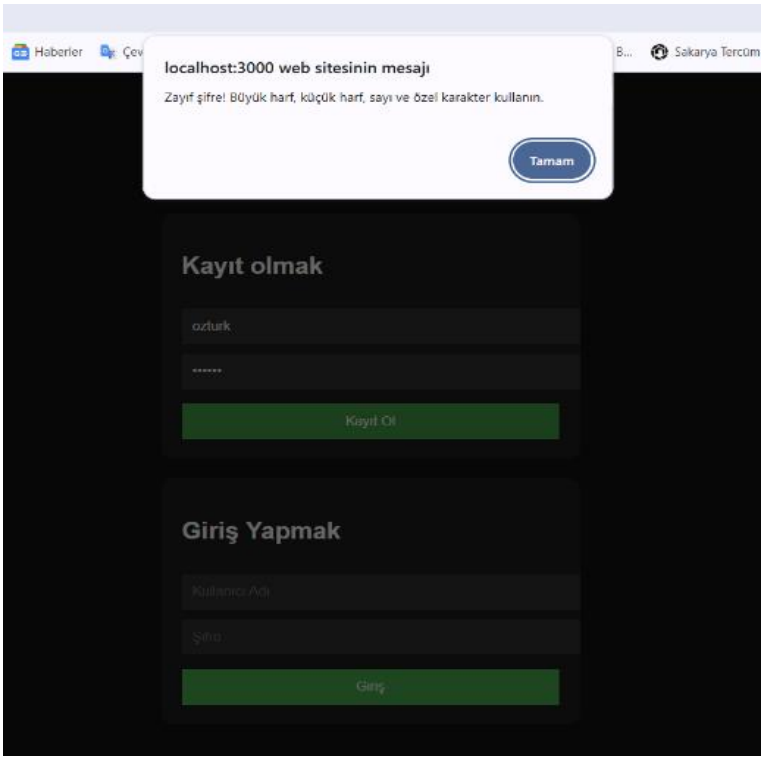


(Şekil 10.1 Hesap Kilitlenmesi)

10.2 Zayıf Şifre Kullanımı

Senaryo:
Kullanıcı zayıf bir şifre ile kayıt olmaya çalışır.

Sistem Tepkisi:
Sistem şifreyi reddeder ve hata mesajı gösterir.

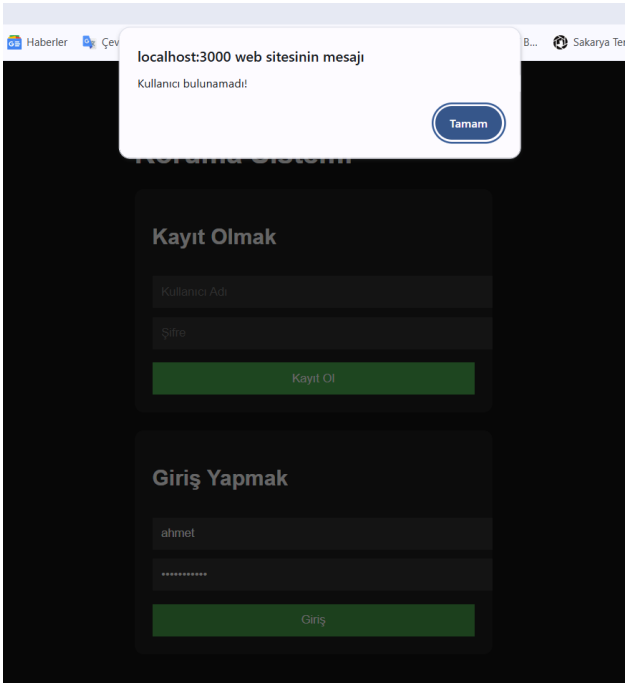


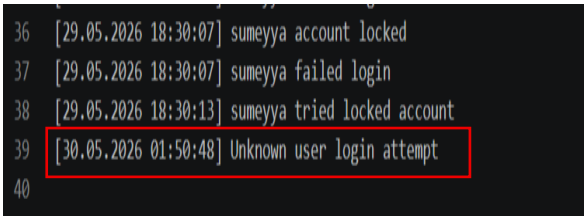
(Şekil 10.2 Hesap Zayıf Şifre Kullanımı)

10.3 Yetkisiz Erişim Girişimi

Senaryo:
Sistemde kayıtlı olmayan bir kullanıcı giriş yapmaya çalışır.

Sistem Tepkisi:
Erişim reddedilir ve işlem log dosyasına kaydedilir.





(Şekil 10.3 Yetkisiz Erişim Girişimi)

11. Güvenlik Testleri

Sistem farklı test senaryoları ile denenmiştir.

Test No	Test Senaryosu	Beklenen Sonuç	Sonuç
1	Başarılı kayıt	Kayıt başarılı	Başarılı
2	Başarılı giriş	Giriş başarılı	Başarılı
3	Yanlış şifre	Hata mesajı	Başarılı
4	Zayıf şifre	Kayıt reddedildi	Başarılı
5	Aynı kullanıcı adı	Kayıt reddedildi	Başarılı
6	Boş şifre	Hata mesajı	Başarılı
7	3 yanlış giriş	Hesap kilitlendi	Başarılı
8	Kilitli hesap girişi	Erişim reddedildi	Başarılı
9	Bilinmeyen kullanıcı	Erişim reddedildi	Başarılı
10	Özel karakterli şifre	Kabul edildi	Başarılı

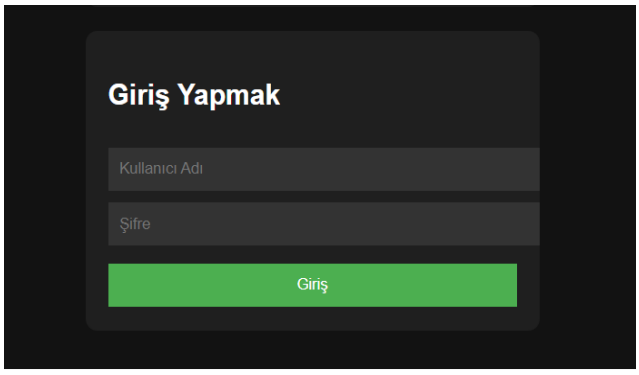
12. Uygulama Çıktıları

Bu bölümde aşağıdaki ekran görüntüleri yer almalıdır:

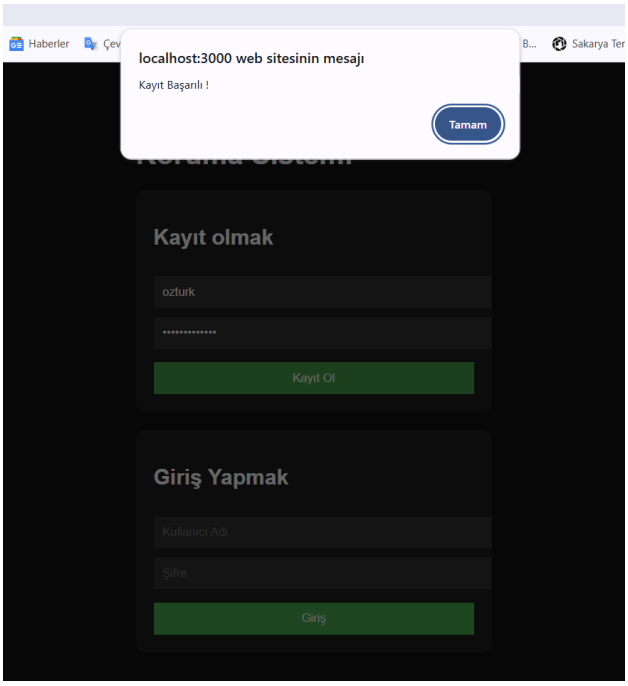
- Kayıt ekranı
- Giriş ekranı
- Başarılı giriş ekranı
- Hata mesajları
- Log dosyası çıktıları



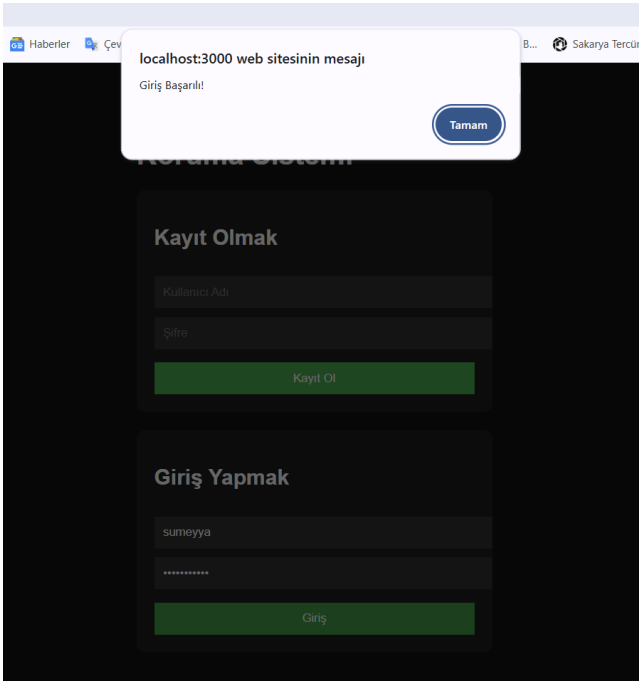
(Şekil 12.1 Kayıt ekranı)



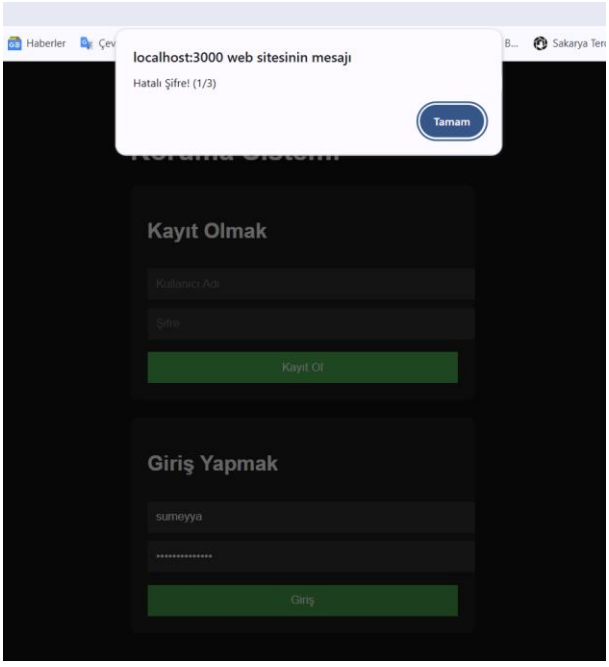
(Şekil 12.2 Giriş ekranı)

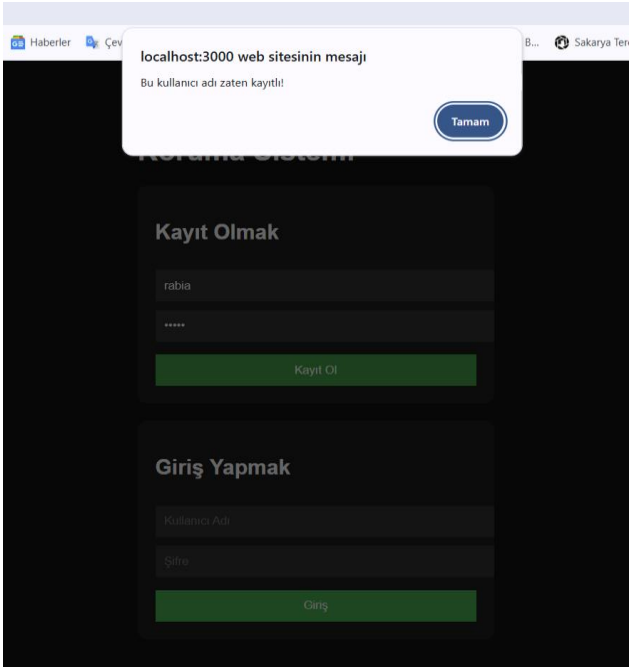


(Şekil 12.3 Kayıt Başarılı)

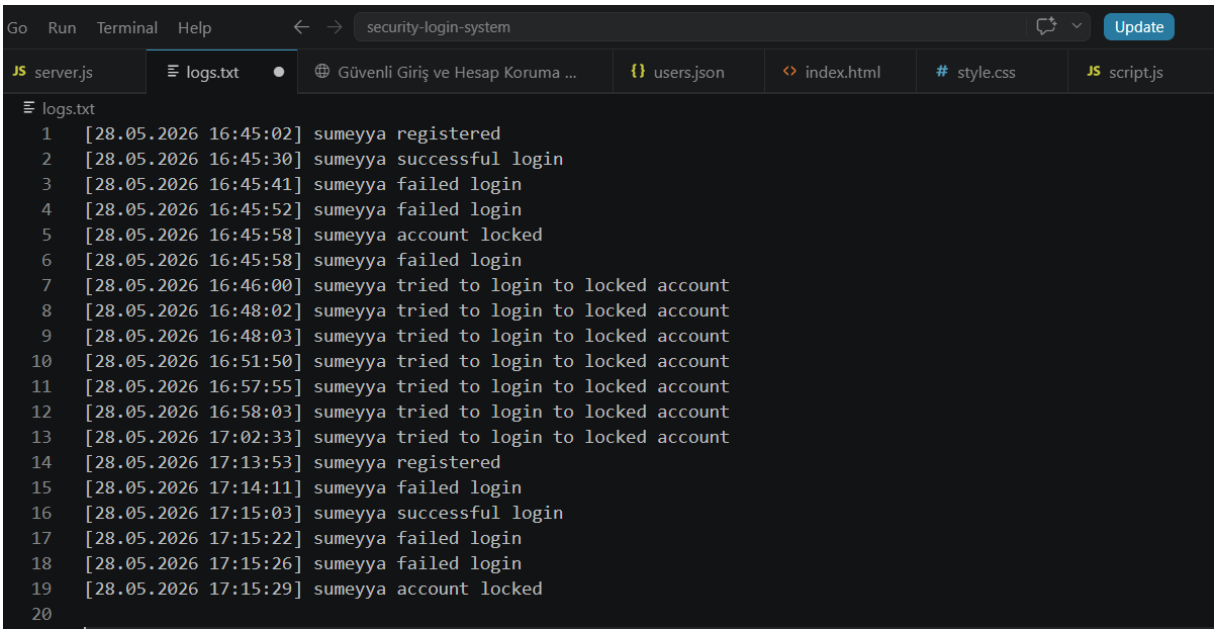


(Şekil 12.4 Giriş Başarılı)





(Şekil 12.5 – 12.6 Hata Mesajları)



(Şekil 12.7 Log Çıktıları)

13. Sonuçlar ve Değerlendirme

Bu projede JavaScript ve Node.js kullanılarak güvenli bir giriş ve hesap koruma sistemi geliştirilmiştir.

Sistem aşağıdaki hedefleri başarıyla gerçekleştirmiştir:

- Güvenli şifre saklama
- Güçlü şifre kontrolü
- Kullanıcı doğrulama
- Saldırı önleme
- Güvenlik loglama

Proje, doğru güvenlik mekanizmaları kullanıldığında basit giriş sistemlerinin bile önemli ölçüde güvenli hâle getirilebileceğini göstermiştir.

Bu proje sayesinde veri güvenliği alanındaki temel kavramlar uygulamalı olarak öğrenilmiştir. Geliştirilen sistem; kimlik doğrulama, hashleme, erişim kontrolü ve saldırı önleme tekniklerini başarılı şekilde göstermektedir.

Ayrıca güvenlik mekanizmalarının yazılım geliştirme sürecinin başından itibaren düşünülmesi gerektiği anlaşılmıştır.

Bu çalışma modern kullanıcı doğrulama sistemlerinin çalışma mantığını anlamak açısından önemli katkı sağlamıştır.

Kaynakça

- William Stallings – Cryptography and Network Security
- Node.js Resmi Dokümantasyonu
- Express.js Dokümantasyonu
- SHA-256 Kriptografi Standardı
- OWASP Authentication Security Guidelines
- JavaScript Crypto Modülü Dokümantasyonu